

Systemy czasu rzeczywistego, Projekt 5

Piotr Gugnowski, wydział EiTI, nr indeksu 320690

Czerwiec 2026

Zadanie 5.1: Standard IEEE 754: zakresy obu systemów kodowania

Treść zadania

Wyznaczyć zakresy liczb w obu systemach kodowania: kodzie "wewnętrznym" oraz w standardzie IEEE 754.

Analiza struktury bitowej

Oba formaty są 32-bitowe. Struktura IEEE 754 single precision: 1 bit znaku, 8 bitów wykładnika z nadmiarem 127, 23 bity mantysy z niejawnym bitem wiodącym.

Struktura kodu wewnętrznego **nie jest wprost podana w treści zadania**. Zadanie 5.1 wymaga jednoznacznego wyznaczenia struktury przez analizę wzorców bitowych (weryfikacja poniżej).

Format IEEE 754 single precision

Bit 31	Bity 30–23	Bity 22–0
znak S (1 bit)	wykładnik ze stałą E_b (8 bitów)	mantysa M (23 bity)

Wartość liczby znormalizowanej:

$$(-1)^S \times (1 + M \cdot 2^{-23}) \times 2^{E_b - 127} \quad E_b \in [1, 254]$$

Stała (bias) wykładnika wynosi 127.

Kod "wewnętrzny"

Na podstawie tabeli konwersji (np. 9 $\rightarrow$ 0x83100000, 65535 $\rightarrow$ 0x8F7FFF00) wyznaczono strukturę bitową:

Bit 31	Bity 30–24	Bit 23	Bity 22–0
"1" (wiodąca)	wykładnik E bez stałej (7 bitów)	"0" (padding)	mantysa M (23 bity)

Wartość liczby niezerowej:

$$1 \times (1 + M \cdot 2^{-23}) \times 2^E \quad E \in [0, 127], \quad M \in [0, 2^{23} - 1]$$

Zero kodowane jest jako 0x00000000.

Kluczowe właściwości kodu wewnętrznego w porównaniu z IEEE 754:

- Brak znaku: format przeznaczony wyłącznie dla liczb nieujemnych.
- Wykładnik **bez stałej** (unbiased): bezpośrednio wyraża potęgę dwójki, co eliminuje konieczność dodawania/odejmowania stałej 127 przy operacjach arytmetycznych.
- Bit 31 jest jawnym bitem wiodącym "1" mantysy (w IEEE 754 bit ten jest niejawnym: tzw. *hidden bit*).
- 7-bitowy wykładnik ogranicza zakres do $2^0 - 2^{127}$, co jest wystarczające dla liczb naturalnych.

Weryfikacja na przykładach z tabeli

Liczba	Kod wewnętrzny	E	M	IEEE 754
0	0x00000000	n/d	n/d	0x00000000
1	0x80000000	0	0	0x3F800000
2	0x81000000	1	0	0x40000000
9	0x83100000	3	0x100000	0x41100000
65535	0x8F7FFF00	15	0x7FFF00	0x477FFF00
65536	0x90000000	16	0	0x47800000

Tabela 1: Weryfikacja struktury bitowej na danych z treści zadania. Dla każdej liczby $E_{\text{IEEE}} = E + 127$.

Przykładowo dla $9 = 1,001_2 \times 2^3$:

$$E = 3, \quad M = 0,001_2 = 2^{-3} \Rightarrow M \cdot 2^{23} = 2^{20} = 0x100000$$

$$E_{\text{IEEE}} = 3 + 127 = 130 = 0x82 \Rightarrow \text{IEEE} = (130 \ll 23) \mid 0x100000 = 0x41100000 \checkmark$$

Zakresy

Kod "wewnętrzny"

Parametr	Wartość
Zakres	$\{0\} \cup [1, (2 - 2^{-23}) \cdot 2^{127}]$
Minimalna wartość niezerowa	$1,0 \times 2^0 = 1$
Maksymalna wartość	$(2 - 2^{-23}) \times 2^{127} \approx 3,403 \times 10^{38}$
Zakres wykładnika E	$[0, 127]$ (7 bitów, bez stałej)
Zakres mantysy M	$[0, 2^{23} - 1]$ (23 bity)
Liczby ujemne	brak
Ułamki < 1	brak

Tabela 2: Zakres kodu "wewnętrznego".

Maksymalna reprezentowalna liczba naturalna (dokładnie):

$$N_{\text{max}} = (2 - 2^{-23}) \times 2^{127} = 2^{128} - 2^{104} \approx 3,403 \times 10^{38}$$

Największa liczba całkowita reprezentowana dokładnie (bez utraty precyzji): format dysponuje 23 bitami mantysy oraz jawnym bitem wiodącym w bicie 31, co daje łącznie 24 bity znaczące. Największa liczba całkowita mieszcząca się w 24 bitach to same jedyńki:

$$N_{\text{int,max}} = \underbrace{11 \dots 1}_{24 \text{ bity}} = 2^{24} - 1 = 16\,777\,215$$

Standard IEEE 754 single precision

Parametr	Wartość
Zakres	$[-(2 - 2^{-23}) \cdot 2^{127}, (2 - 2^{-23}) \cdot 2^{127}]$
Minimalna wartość dodatnia	$2^{-126} \approx 1,175 \times 10^{-38}$
Maksymalna wartość	$(2 - 2^{-23}) \times 2^{127} \approx 3,403 \times 10^{38}$
Zakres wykładnika	$E \in [-126, +127]$ (bias = 127)
Zakres mantysy M	$[0, 2^{23} - 1]$ (23 bity)
Liczby ujemne	tak (bit znaku)

Tabela 3: Zakres standardu IEEE 754 single precision (32-bit).

Porównanie zakresów

Cecha	Kod wewnętrzny	IEEE 754
Zakres dodatni	$[1, (2 - 2^{-23}) \cdot 2^{127}]$	$[2^{-126}, (2 - 2^{-23}) \cdot 2^{127}]$
Wartości < 1	nie	tak
Liczby ujemne	nie	tak
Bity wykładnika	7 (bez stałej)	8 (stała 127)
Bity mantysy	23	23

Tabela 4: Porównanie cech obu systemów kodowania.

Oba formaty mają identyczną wartość maksymalną i identyczne pole mantysy (23 bity). Różnica leży w zakresie: IEEE 754 obsługuje ułamki i liczby ujemne, kod wewnętrzny ogranicza się do liczb całkowitych ≥ 1 . W zamian kod wewnętrzny jest prostszy sprzętowo: wykładnik bez stałej i jawny bit wiodący eliminują dodatkowe operacje przy każdym obliczeniu.

Zadanie 5.2: Konwersja kodu wewnętrznego do IEEE 754

Treść zadania

Napisać aplikację konwersji kodu "wewnętrznego" liczb zmiennoprzecinkowych do kodu liczb zmiennoprzecinkowych zgodnych z IEEE 754. Aplikacja ma umożliwiać wprowadzenie z klawiatury liczb w kodzie binarnym naturalnym.

Implementacja

Aplikacja napisana w C++17, kompilowalna na systemach Windows i macOS (clang++ lub g++). Do budowania wymagany jest `just`. W folderze projektu znajduje się skompilowana wersja dla systemu Windows. Kod źródłowy składa się z dwóch plików:

- `src/shared.cpp` / `src/shared.h`: funkcje wspólne (konwersje, wyświetlanie, parsowanie wejścia),
- `src/task2_internal_to_ieee.cpp`: punkt wejścia (`main`).

Format wejścia

Zgodnie z założeniem zadania użytkownik wprowadza kod wewnętrzny jako **32-bitowy ciąg w kodzie binarnym naturalnym** (ciąg znaków 0/1). Spacje są dozwolone i ignorowane, co umożliwia grupowanie bitów dla czytelności.

Przykład dla liczby 9, której kod wewnętrzny to `0x83100000`:

```
$ ./task2_internal_to_ieee 10000011000100000000000000000000
$ ./task2_internal_to_ieee "1000 0011 0001 0000 0000 0000 0000 0000"
```

Aplikacja obsługuje dwa tryby:

1. **Argument wiersza poleceń**: 32-bitowy ciąg binarny podany bezpośrednio.
2. **Tryb interaktywny**: aplikacja wyświetla prompt i czeka na wejście.

Algorytm konwersji: kod wewnętrzny $\rightarrow$ IEEE 754

Konwersja sprowadza się do trzech operacji bitowych:

1. Wyodrębnienie wykładnika bez stałej E z bitów $[30:24]$.
2. Dodanie stałej 127: $E_b = E + 127$.
3. Złożenie słowa IEEE 754: bit znaku = 0, wykładnik = E_b , mantysa bez zmian.

```
1 uint32_t internal_to_ieee754(uint32_t internal) noexcept {
2     if (internal == 0u) return 0u;
3     const uint32_t E = (internal >> 24u) & 0x7Fu;
4     const uint32_t mant = internal & 0x007FFFFFFu;
5     const uint32_t E_ieee = E + 127u;
6     return (E_ieee << 23u) | mant;
7 }
```

Wyniki dla danych wzorcowych

Liczba	Wejście (kod binarny naturalny)	Kod wewn. (hex)	IEEE 754 (hex)
0	0000...0 (32 zera)	0x00000000	0x00000000
1	1000 0000 0000 ... 0000 0000	0x80000000	0x3F800000
9	1000 0011 0001 0000 0000 0000 ...	0x83100000	0x41100000
65535	1000 1111 0111 1111 1111 1111 ...	0x8F7FFF00	0x477FFF00
65536	1001 0000 0000 ... 0000 0000	0x90000000	0x47800000

Tabela 5: Wyniki konwersji: wejście w kodzie binarnym naturalnym, wyjście IEEE 754.

Zadanie 5.3: Konwersja IEEE 754 do kodu wewnętrznego

Treść zadania

Napisać aplikację konwersji liczb zmiennoprzecinkowych zgodnych z IEEE 754 do postaci kodu "wewnętrznego".

Implementacja

Aplikacja napisana w C++17, kompilowalna na systemach Windows i macOS (clang++ lub g++). Kod źródłowy:

- `src/shared.cpp` / `src/shared.h`: funkcje wspólne (te same co w zadaniu 5.2),
- `src/task3_ieee_to_internal.cpp`: punkt wejścia (`main`).

Format wejścia

Użytkownik wprowadza **32-bitową wartość IEEE 754 w postaci szesnastkowej** (z przedrostkiem `0x` lub bez). Aplikacja odczytuje wzorzec bitowy słowa i konwertuje go na kod wewnętrzny, wypisując wynik również w postaci szesnastkowej.

Przykłady wywołania:

```
$ ./task3_ieee_to_internal 0x41100000
$ ./task3_ieee_to_internal 477FFF00
$ ./task3_ieee_to_internal 0x3F800000
```

Aplikacja obsługuje dwa tryby:

1. **Argument wiersza poleceń**: wartość szesnastkowa podana bezpośrednio.
2. **Tryb interaktywny**: aplikacja wyświetla prompt i czeka na wejście.

Walidacja wejścia

Przed konwersją sprawdzane są warunki konieczne, gdyż zakres kodu wewnętrznego jest podzbiorem zakresu IEEE 754:

Warunek	Komunikat błędu
Bit 31 = 1 (liczba ujemna)	liczba ujemna: niedozwolona
$E_b = 255$ (Inf lub NaN)	Inf/NaN: brak reprezentacji
$E_b = 0$ (zdenormalizowana)	subnormal: poza zakresem
$E = E_b - 127 < 0$	wartość < 1: poza zakresem
$E > 127$	wykładnik > 127: poza zakresem

Algorytm konwersji: IEEE 754 → kod wewnętrzny

```
1 uint32_t ieee754_to_internal(uint32_t ieee) noexcept {
2     if (ieee == 0u) return 0u;
3     // ... (walidacja jak w tabeli powyżej) ...
4     const uint32_t E_biased = (ieee >> 23u) & 0xFFu;
5     const uint32_t mant = ieee & 0x007FFFFFu;
6     const int32_t E = static_cast<int32_t>(E_biased) - 127;
7     return 0x80000000u | (static_cast<uint32_t>(E) << 24u) | mant;
8 }
```

Operacja jest dokładnie odwrotna do zadania 5.2: zamiast dodawać stałą 127 odejmujemy ją; zamiast zerować bit 31 ustawiamy go na 1. Mantysa (bity 22:0) pozostaje niezmienną w obu kierunkach.

Wyniki dla danych wzorcowych

Liczba	IEEE 754 (wejście)	Kod wewnętrzny (wyjście)
0	0x00000000	0x00000000
1	0x3F800000	0x80000000
2	0x40000000	0x81000000
9	0x41100000	0x83100000
65535	0x477FFF00	0x8F7FFF00
65536	0x47800000	0x90000000

Tabela 6: Wyniki konwersji IEEE 754 $\rightarrow$ kod wewnętrzny. Wejście i wyjście w postaci szesnastkowej.